İSTANBUL DATA SCIENCE ACADEMY

Copilot, MCP ve Token Ekonomisi

Çalışma kitabı

Bağlam bir bütçedir. Bu kitap günün akışını izler: her bölümde önce **konunun özeti**, hemen ardından **o konunun labı**. Okuyun, uygulayın, sonraki bölüme geçin.

EĞITMEN

Zekeriya Beşiroğlu

Zekeriya.besiroglu@istdsa.com

Bir günlük atölye
2026

İçindekiler

00 Nasıl kullanılır ve günün akışı 3
Kitabın yapısı, saat saat program

01 Bölüm 1 — Ne gönderiyorsunuz 4
Talimat dosyaları: hangi kural hangi dosyaya, ve her satırın bedeli

02 Bölüm 2 — Neyi açıyorsunuz 7
MCP: kurum içi kaynağı araç olarak açmak ve sınırlarını çizmek

03 Bölüm 3 — Ne ödüyorsunuz 11
Token ekonomisi: sayaç, krediler, havuzlanma ve üç eşik

04 Bölüm 4 — Optimizasyon 15
Yedi kaldıraç, hangisi kural hangisi alışkanlık

05 Referans kartı 18
Günün tamamı tek sayfada

06 Yarın uygulayabileceğiniz üç şey 19
İzin gerektirmeyen üç değişiklik

Bu kitap nasıl kullanılır

Konu özetleri anlatım sırasında takip edilir, sonrasında tekrar okunur. **Lab adımlarında** her adımda üç şey var: ne yaptığınız, komut ve **görmeniz gereken çıktı**. Gördüğünüz çıktı yazandan farklıysa devam etmeyin, o adımda kalın.

Kendinize sorun bölümlerindeki cevaplar kapanış atölyesinde kullanılacak — boş bırakmayın.

Kurulum bu kitapta yok: eğitimden önce reponun içindeki sıfırdan yapma kılavuzuyla (VS Code veya IntelliJ) ortamınızı hazırlayın.

BAŞLANGIÇ

Nasıl kullanılır ve günün akışı

Kitabın yapısı ve saat saat program

Copilot, MCP ve Token Ekonomisi · bir günlük atölye

> Bağlam bir bütçedir. Ne gönderdiğiniz, neyi açtığınız, ne ödediğiniz.

Bu kitap günün akışını izler: her bölümde önce konunun özeti, hemen ardından o konunun labı. Okuyun, uygulayın, sonraki bölüme geçin.

Nasıl kullanılır

Konu özetleri	Anlatım sırasında takip edin, sonrasında tekrar okuyun
Lab adımları	Komutları kopyalayıp çalıştırın, beklenen çıktıyla karşılaştırın
Kendinize sorun	Her bölüm sonunda; cevaplar kapanış atölyesinde işinize yarayacak

Her lab adımında üç şey var: ne yaptığınız, komut ve **görmeniz gereken çıktı**. Gördüğünüz çıktı yazandan farklıysa devam etmeyin.

Kurulum bu kitapta yok. Eğitimden önce `SIFIRDAN-YAPMA-KILAVUZU.md` (VS Code) veya `labs/intellij/SIFIRDAN-YAPMA-KILAVUZU.md` (IntelliJ) ile ortamınızı hazırlayın.

Günün akışı

SAAT	BÖLÜM	SÜRE
09:30–11:00	`1 · Ne gönderiyorsunuz`	Özet 45 dk + Lab 45 dk
11:15–12:45	`2 · Neyi açıyorsunuz`	Özet 30 dk + Lab 60 dk
13:45–15:00	`3 · Ne ödüyorsunuz`	Özet 30 dk + Lab 45 dk
15:15–16:30	`4 · Optimizasyon`	Özet 15 dk + Lab 60 dk
16:30–17:00	Kapanış: politika taslağı	Grup çalışması

BÖLÜM 1

Ne gönderiyorsunuz

Talimat dosyaları: hangi kural hangi dosyaya, ve her satırın bedeli

Konu özeti

Copilot'un davranışını repodaki dosyalarla yönetebiliyorsunuz. Dört dosya türü var ve aralarındaki fark *ne yaptıkları* değil, **ne zaman yüklendikleri**.

DOSYA	NE ZAMAN YÜKLENIR	NE IÇIN
`copilot-instructions.md`	Her istekte	Her yerde geçerli değişmezler
`*.instructions.md`	`applyTo` eşleşince	Teknolojiye özgü ayrıntı
`*.prompt.md`	`/komut` yazınca	Tekrarlanan çok adımlı görevler
`*.chatmode.md`	Mod seçilince	Farklı çalışma kipi, kısıtlı araçlar

> **Karar kuralı:** her soruda gerekli değilse, her istekte yüklenen dosyaya girmez.

Her satırın bir bedeli var

`copilot-instructions.md`'ye yazdığınız her satır, ekipteki herkesin sorduğu her soruya ekleniyor. Somut hesap:

120 kişi × günde 4 istek × 21 gün × 1.000 fazla token	**10,1 milyon token/ay**
Hafif modelde	$2/ay · $24/yıl
Dengeli modelde	$20/ay · $242/yıl
Güçlü modelde	$50/ay · $605/yıl

Rakam, görmezden gelinecek kadar küçük ve utandıracak kadar büyük. Bu dosyaların yıllarca kabararak hayatta kalmasının sebebi tam olarak bu.

Satır satır denetim

Her satır için tek bir soru: *ekibimin sorduğu soruların yüzde kaçında bu satır gerçekten gerekli?*

CEVAP	SATIR NEREYE AIT
%80 ve üstü	Burada kalır
%20 – %80	`instructions/` altına, `applyTo` ile
%20 altı	`.prompt.md` dosyasına

Hedef: her istekte yüklenen dosya 500 token altında.

applyTo — kapsamın maliyeti düşürmesi

Kapsamlı dosyaların başında bir glob var: `applyTo: "**/*.java"`. Bu kural yalnızca bir `.java` dosyasına dokunulduğunda yükleniyor.

Burada bir takas **yok**: kapsamlama hem ucuzlatıyor hem kaliteyi yükseltiyor. Bağlamda daha az alakasız kural olması, modelin yanlış kuralı uygulama ihtimalini düşürüyor.

> **Bu dosya kod mu, politika mı?**

Dosya kod gibi davranıyor — repoda duruyor, PR'la değişiyor. Ama politika gibi sonuç doğuruyor: bir kişinin eklediği satır, herkesin her sorusunun maliyetine giriyor.

Çoğu kurumda üç sorunun cevabı verilmemiş: kim değiştirebilir, hangi incelemeden geçer, ne zaman yeniden okunur.

Lab 1 · 45 dakika

1.1 Önce ölçün

Hiçbir dosyaya bakmadan sohbete sorun:

```
Bu projede para tutarları için hangi tipi kullanmalıyım?
```

Görmeniz gereken: genel bir cevap. Not edin.

> Talimat dosyasını önce eklerseniz farkı hiç göremezsiniz. **Önce ölç, sonra değiştir** – bugün yapacağınız her ölçümde bu sıra geçerli.

1.2 Talimat dosyasını okuyun, tekrar sorun

`.github/copilot-instructions.md` dosyasını açıp okuyun. Sonra **yeni bir sohbette** aynı soruyu sorun.

Görmeniz gereken: `BigDecimal`, asla `double`.

UYARI

Yeni sohbet şart. Aynı sohbette sorarsanız ilk cevap hâlâ bağlamda durur ve ikincisi kirlenir. Lab 3'te aynı hata bütün ölçümünüzü bozar.

1.3 Dosyaları tartın

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import estimate_tokens
from pathlib import Path
t=0
for f in sorted(Path('.github').rglob('*.md')):
    n=estimate_tokens(f.read_text()); t+=n
    print(f'{n:>6} token  {f}')
print(f'{t:>6} token  TOPLAM')
"
```

Görmeniz gereken: `copilot-instructions.md` ≈ 292 token, toplam ≈ 1712.

Şimdi kendi kurumunuzun dosyasını aynı komutla ölçün. Çoğu kurumda rakam üç ila on kat büyük çıkıyor.

1.4 applyTo farkını görün

`.github/instructions/java-service.instructions.md` dosyasının başındaki `applyTo` satırına bakın. Sonra iki test — **her biri ayrı yeni sohbette**:

1. Bir `.py` dosyası açın, sorun: *Buradaki katman kuralları neler?*
2. Bir `.java` dosyası açın, **aynı** soruyu sorun

Görmeniz gereken: ikinci cevap Controller/Service/Repository katmanlarından bahsediyor, birincisi bahsetmiyor.

> INTELLIJ KULLANIYORSANIZ
>
> JetBrains `applyTo` desteklemiyor — `instructions/` klasörü okunmuyor. Bu adım yerine: dosyayı açıp içeriğin nereye ait olduğuna karar verin. Hepsi her isteğe mi girmeli, yoksa bir kısmı prompt dosyasına mı taşınmalı? Kararınızı bir cümleyle yazın, Lab 4'te uygulayacaksınız.
>
> Ayrıntı: `labs/intellij/LAB-1-talimat-dosyalari.md`

1.5 Prompt dosyasını deneyin ve kendinizinkini yazın

Bir `.java` dosyası açıkken sohbete `/service-review` yazın.

Sonra haftada tekrarladığınız bir işi seçip `.github/prompts/` altına kendi dosyanızı yazın. Üç kural: **çok adımlı**, **açık çıktı biçimi**, **sonunda bir duruş** ("önce raporu ver, onay bekle").

1.6 Analist modunu deneyin

Rol seçicisinden **analyst** modunu seçin (IntelliJ'de: sohbete `/analist` yazın). Sonra sırayla:

```
Sürüm takvimi kuralları nerede tanımlı ve ne diyor?
```

```
Bana bir servis sınıfı yaz.
```

Görmeniz gereken: ikincisini reddetmeli.

Kendinize sorun

1. Kurumunuzda bu dosyaları kim yazar ve kim onaylar? Bir geliştirici bir satır eklediğinde maliyet herkese yayılıyor. **Bu dosya kod mudur, politika mıdır?**
2. Kendi `copilot-instructions.md` dosyanızda kaç satır gerçekten %80'in üstünde gerekli? Kaçı taşınmalı?

	NOTUNUZ
Dosyamın token sayısı	
Taşınacak satır sayısı	

BÖLÜM 2

Neyi açıyorsunuz

MCP: kurum içi kaynağı araç olarak açmak ve sınırlarını çizmek

Konu özeti

MCP (Model Context Protocol), modelin dış dünyayla konuşması için standart bir arayüz. Üç şey sunuyor: **tools** (çağrılabilir fonksiyonlar), **resources** (okunabilir içerik), **prompts** (hazır şablonlar).

ÖNEMLI

Copilot yalnızca tools kullanıyor. Resources ve prompts desteklenmiyor — ve size bunu kimse söylemez. Hata almazsınız, sadece beklediğiniz şey olmaz.

Bankada üç kritik nokta

Bir — yerel çalışmak, içeride kalmak değildir.

Zinciri üç adımda görün:

1. Aracın döndürdüğü her şey bağlama giriyor — girmek zorunda, model onun üzerinde akıl yürütecek
2. Bağlam model sağlayıcısına gidiyor — bu turda ve sonraki her turda
3. Dolayısıyla araç bir **veri yolu**

Bir MCP aracı, kimse buna karar vermeden sınırınızın dışına veri taşıyan bir kanala dönüşebilir. Bu kurumsal bir karardır, geliştirici kararı değildir.

İki — onay akışı denetim izidir.

SEÇENEK	NE ELDE EDERSINIZ
Her çağrıda onay	Her kullanım, bir insanın belirli bir anda verdiği karar
Otomatik onay	Kararların kaydı, faaliyetin kaydına dönüşür
Orta yol	Salt-okunur araçlar otomatik; yazanlar asla

Üç — onaylı liste, serbest kurulum değil.

Bir onayın yanıtlaması gereken üç soru: **ne döndürüyor** (hangi alanlar, kişisel veri var mı), **kim sahibi** (araç yanlış şey döndürdüğünde cevap verecek ekip), **kim bağlayabilir**.

Araç bütçesi

Bağlı olan her sunucu, araç şemalarını **her isteğe** ekliyor — o sunucuyu o soruda kullanıp kullanmamanız fark etmiyor.

BAĞLI SUNUCU	HER ISTEĞE EKLENEN ŞEMA (8 ARAÇ/SUNUCU)
1 sunucu	8
3 sunucu	24
10 sunucu	80

Copilot'un tavanı istek başına 128 araç, ama kalite bozulması çok daha önce başlıyor — model bunlar arasından *seçmek* zorunda.

> **Kural:** bugün kullanmadığınız sunucuyu kapatın.

İyi araç tasarımı

Model araç seçimini **açıklamaları okuyarak** yapıyor:

KÖTÜ	İYI
"Servis sahibi bilgisini döndürür."	"Bir servisten sorumlu ekip gerektiğinde kullan — kayıt açmadan veya olayı yükseltmeden önce. Kısmi isim varsa önce `api_search` çağır."

Üç kural: **faydalı biçimde başarısız olun** (mevcut seçenekleri döndürün), **gerekçeyi de döndürün** (modelin okuduğu gerekçe, uyduramayacağı gerekçedir), **açıklamaları kısa tutun** (her kelime her istekte gidiyor).

Lab 2 · 60 dakika

2.1 Sunucuyu elle bir kez çalıştırın

IDE'ye dokunmadan önce:

```
"$(pwd)/.venv/bin/python" "$(pwd)/mcp_server/corp_server.py"
```

Görmeniz gereken: hiçbir şey — terminal donar ve bekler. Bu doğru. `Ctrl+C`.

DIKKAT

Traceback dökerse hata buradadır. IDE'nin sizden saklayacağı şeyi burada görüyorsunuz.

2.2 Yapılandırın ve başlatın

VS Code: `.vscode/mcp.json` dosyasına yorumlayıcının **tam yolunu** yazın, sonra `MCP: List Servers` → `corp-internal` → Start Server.

IntelliJ: Copilot ikonu → Edit Settings → Model Context Protocol → Configure. Yolları elle yazın.

DIKKAT

Düz `python` yazmayın. PATH'teki ilk yorumlayıcı çalışır, `mcp` paketi orada olmadığı için süreç anında ölür ve **sebebi görünmez**.

2.3 Araçları doğrulayın

Sohbete `#` yazın — beş araç listelenmeli. Çıkmıyorsa doğrudan sorun:

```
standards_list aracını çalıştır
```

Görmeniz gereken: "Allow tool from corp-internal" istemi. Bu istem geldiyse her şey çalışıyor.

2.4 Üç soruyu sorun

Her birini **ayrı sohbette**:

```
Hangi dahili API'ler kişisel veri içermiyor?
```

```
Bugün üretime çıkabilir miyiz?
```

```
14 Ağustos 2026'da deploy edebilir miyiz?
```

Görmeniz gereken: üçüncüsü **hayır** demeli ve gerekçesini de vermeli.

> Araç sadece `false` döndürmüyor, nedenini de döndürüyor. Model gerekçeyi uyduramıyor, alıntılamak zorunda.

2.5 Kendi kolunuz

Geliştiriciyseniz — `find_service_owner` aracını yazın, testiyle birlikte. Docstring "ne zaman çağrılacağını" söylesin; bulunamayınca mevcut seçenekleri döndürsün; kişisel veri döndürmesin. `python -m pytest tests -q` → **30 passed**.

Analistseniz — beş docstring'i okuyun ve dört soruya yazılı cevap verin:

1. Hangi açıklama modelin **yanlış aracı** seçmesine yol açabilir?
2. `api_search` aracının `pii_free_only` parametresi olmasaydı ne bozulurdu?
3. Bu sunucuya **eklenmemesi** gereken bir araç söyleyin, nedenini savunun
4. `can_deploy` sadece `true` / `false` dönseydi ne kaybederdik?

> Bu kolay kol değil. Bankada asıl kritik soru bir aracın *yazılmaması* gerektiği, ve bu soru Python bilene değil iş sonucunu bilene ait.

2.6 Sunucunun maliyetini görün

```
tool_cost aracını çalıştır
```

Hesabı yapın: sekizer araçlı on sunucu, her sorunun başına seksen şema ekler.

Kendinize sorun

1. Bir ekip müşteri sorgulama aracı yazmak isterse **hangi onaylardan geçmeli** ve kararı kim verir: geliştirici mi, mimari mi, uyum mu?
2. Bir denetçi *"geçen ay hangi veri bir araç çağrısı üzerinden sınırın dışına çıktı"* diye sorsa, kurumunuzda kim cevap verebilir?

	NOTUNUZ
Kurumumuzda MCP onayını kim verir	
Denetim sorusunun cevabı var mı	

BÖLÜM 3

Ne ödüyorsunuz

Token ekonomisi: sayaç, krediler, havuzlanma ve üç eşik

Konu özeti

Üç şey sayılıyor: **girdi** (gönderdiğiniz her şey, her turda), **çıktı** (modelin yazdığı, girdiden birkaç kat pahalı), **önbellekli girdi** (değişmeyen bağlam, onda bir fiyata).

Her şey tek bir dönüşüme iniyor: **1 AI kredisi = 1 sent.**

ÖRNEK BIR ISTEK	
Girdi — 100.000 token × $2,00/1M	$0,20
Çıktı — 3.000 token × $12,00/1M	$0,04
Toplam	**$0,24 = 24 kredi**

Tek istek önemsiz. 120 kişi × 4 istek × 21 gün değil. Birim küçük, çarpan kadronuz.

Havuzlanma

Koltuk başına dahil kredi Business'ta $19, Enterprise'da $39. Ama havuz koltuk başına değil — **fatura varlığı düzeyinde toplanıyor**. 120 koltuklu bir kurumda aylık havuz $2.280.

Pratikte: bir ağır kullanan ekip, birkaç hafif ekibin başlığını tüketebilir — ve araçlarda hiçbir şey bunu göstermiyor, fatura gelene kadar.

Üç eşik

EŞIK	KAT	NEREDEN GELIYOR
Model seçimi	25×	Hafif $0,0236 ↔ güçlü $0,5900 (aynı istek)
Önbellek	10×	Değişmeyen bağlam onda bir fiyata
Uzun bağlam	2×	272K aşılınca birim fiyat ikiye katlanıyor

UYARI

Önbellek bedava değil. Anthropic modellerinde önbelleğe **yazmak** düz girdiden pahalı (Sonnet 5: girdi $2,00, önbellekli $0,20, yazma $2,50). Önbellek bir yatırım — tekrar kullanımda geri dönüyor, kısa ömürlü bağlamda zarar bile ettirebiliyor.

Kaldıraçlar çarpılıyor

DEĞIŞEN	MALIYET	KAZANÇ
Hiçbiri	$0,5110	—
Yalnız model hafifledi	$0,0204	25×
Yalnız bağlam küçüldü	$0,0570	9×
İkisi birden	**$0,0023**	**224×**

Kaba hesap 25 × 9 = 225, ölçülen 224×. **Kaldıraçlar toplanmıyor, çarpılıyor.**

Lab 3 · 45 dakika

3.1 Gerçek rakamınızı bulun

GitHub → Settings → Billing → Usage

	SIZIN RAKAMINIZ
Bu ay harcanan AI Credits	
Karışımdaki modeller	
Dahil kredinin yüzde kaçı	

DIKKAT

Bu adımı atlamayın. Sonraki her şey bu rakama dayanıyor. *Tahminle bütçe kurmak, ölçmemenin kibarcasıdır.*

3.2 Senaryo scriptini çalıştırın ve kalibre edin

```
python cost/scenarios.py
```

Sonra `cost/scenarios.py` içindeki üç sabiti kendi ekibinize göre değiştirin:

```
SEATS = 120          # kendi ekip büyüklüğünüz
WORKDAYS = 21
ALLOWANCE = 19.0     # Business 19, Enterprise 39
```

Tekrar çalıştırıp **3.1'deki gerçek rakamla karşılaştırın.** Fark büyükse hatalı olan hesaplayıcı değil, sizin kullanım varsayımlarınız.

3.3 Üç eşiği doğrulayın

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import Interaction
for m in ('gpt-5.6-luna','gpt-5.6-terra','gpt-5.6-sol'):
    i = Interaction(m, input_tokens=100_000, output_tokens=3_000)
    print(f'{m:>16}  \${i.cost_usd():.4f}  {i.credits():>6.1f} kredi')
"
```

Görmeniz gereken:

```
    gpt-5.6-luna  $0.0236     2.4 kredi
   gpt-5.6-terra  $0.2360    23.6 kredi
     gpt-5.6-sol  $0.5900    59.0 kredi
```

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import Interaction
tam = Interaction('gpt-5.6-terra', input_tokens=100_000)
onb = Interaction('gpt-5.6-terra', cached_input_tokens=100_000)
print(f'oran {tam.cost_usd()/onb.cost_usd():.1f}x')
"
```

Görmeniz gereken: `oran 10.0x`

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import Interaction
for n in (270_000, 280_000):
    i = Interaction('gpt-5.6-terra', input_tokens=n, output_tokens=3_000)
    print(f'{n:>9,} token → \${i.cost_usd():.4f}')
"
```

Görmeniz gereken: `$0.5760` ve `$1.1740` — %3,7 daha çok token, %104 daha çok para.

3.4 Kendinizi ölçün (A/B koşusu)

Bir görev seçin. İki kez yapın:

- **A:** beş dosya bağlamda, güçlü model, uzun sohbetin ortasında
- **B:** yeni sohbet, tek dosya, hafif model, net istek

	A KOŞUSU	B KOŞUSU
Kullanılan model		
Bağlamdaki dosya sayısı		
Harcanan kredi		
Cevap kabul edilebilir mi?		

ÖNEMLI

Dördüncü satır boşsa ölçüm geçersizdir. B'nin cevabını gerçekten kullanır mıydınız, yoksa A'yı görmüş olduğunuz için mi kabul edilebilir göründü?

Kendinize sorun

1. Havuzlanmış kredide bir ekibin aşımı herkesin sorunu. **Bütçe kimin olmalı** — ekip lideri mi, platform ekibi mi?
2. Havuz aşıma girdiğinde **ilk hangi veri** istenmeli?

	NOTUNUZ
Bizim ekibin aylık tahmini	
Kullanım raporuna kim erişebiliyor	

BÖLÜM 4

Optimizasyon

Yedi kaldıraç, hangisi kural hangisi alışkanlık

Konu özeti

Yedi kaldıraç var ve sıra önemli: listeyi yukarıdan aşağı uygulayın, çünkü ilk üçü tasarrufun çoğunu veriyor.

#	KALDIRAÇ	NE DEMEK
1	Doğru yüzey	Tamamlama ölçülmüyor; o işi sohbete taşımak bedavayı ücretliye çevirir
2	Model yönlendirme	Aynı istekte 25 kata kadar fark
3	Bağlam disiplini	Eklenen her dosya her turda yeniden gidiyor
4	Önbellek sıralaması	Sabit kısmı sabit tutun
5	Araç bütçesi	Kullanmadığınız sunucuyu kapatın
6	Yeni sohbet refleksi	Konu değiştiyse baştan başlayın
7	Yeniden deneme	İyi yazılmış tek istek, üç kötüden ucuz

İlk üçünün tetiği

Alışkanlığın somut bir tetiği olmalı, yoksa yerleşmiyor:

KALDIRAÇ	TETIK	ONUN YERINE
Doğru yüzey	Kodu sohbete yapıştırmak üzeresiniz	Satır içi yapın
Model yönlendirme	Modeli düşünmeden sohbet açıyorsunuz	Hafifle başlayın, yetmezse yükseltin
Bağlam disiplini	Dosya yerine klasör ekliyorsunuz	Sorunun ihtiyacı kadarını ekleyin

Kural mı, alışkanlık mı?

Yanlış mekanizma ikisini birden harcar.

KALDIRAÇ	MEKANIZMA
2 · Model yönlendirme	Varsayılan ayar
4 · Önbellek sıralaması	Politika
5 · Araç bütçesi	Politika + onaylı liste
1, 3 · Yüzey ve bağlam	Eğitim + örnek
6, 7 · Yeni sohbet, yeniden deneme	Yalnızca alışkanlık

"Yeni sohbet aç" diye bir politika yazarsanız iki şey olur: kural işlemez ve kuralı yazan kişi itibar kaybeder. **Denetlenemeyen bir kural, kural değil temennidir.**

> **Ucuzlatmak hedef değil**

> **İşi yapamayan bir kurulum üreten optimizasyon, optimizasyon değildir.**

Başarısızlık biçimi: fatura düşüyor, kabul oranı da düşüyor, insanlar işi elle yapmaya dönüyor — ve kimse raporlamıyor, çünkü herkesin baktığı metrik iyi görünüyor.

Kontrol basit: **kabul edilebilirliği maliyetle aynı tabloya yazın.**

Lab 4 · 60 dakika

4.1 Her dosyayı tartın

```
python -c "
import sys; sys.path.insert(0,'.')
from cost.calculator import estimate_tokens
from pathlib import Path
toplam = 0
for f in sorted(Path('.github').rglob('*.md')):
    t = estimate_tokens(f.read_text()); toplam += t
    print(f'{t:>6} token  {f}')
print(f'{toplam:>6} token  TOPLAM')
"
```

Şimdi **kendi reponuzda** çalıştırın. O rakam bugün üzerinde çalışacağınız şey.

4.2 Always-on dosyayı kesin

Hedef: 500 token altı. Her satıra Lab 1'deki testi uygulayın.

İÇERIK	NEREYE
Teknolojiye özgü kurallar	`instructions/` + `applyTo` (IntelliJ'de: prompt dosyası)
Kod inceleme kontrol listesi	`.prompt.md`
Onboarding metni	Wiki — model bunu okumuyor bile

Tasarrufu paraya çevirin:

KESILEN	AYLIK TOKEN	DENGELI MODEL	YILLIK
500 token	5,04M	$10,08	$121
1.000 token	10,08M	$20,16	$242
2.000 token	20,16M	$40,32	$484

DIKKAT

Sahte sonucun işareti: dosyayı küçülttünüz ama hiçbir satırı **taşımadıysanız**, muhtemelen gerçekten gerekli kuralları sildiniz.

4.3 Model yönlendirme tablonuzu yazın

İŞ TÜRÜ	MODEL SINIFI	GEREKÇE
Satır içi tamamlama	varsayılan	Hiç ölçülmüyor
Basit soru, kod okuma	hafif	Muhakeme gerekmiyor
Hata ayıklama, refactor	dengeli	Günlük işin dürüst ortası
Mimari, karmaşık agent işi	güçlü	Fiyatını hak ediyor
Kod incelemesi	seçici	Koşu başına pahalı

İyi bir tablo: **model adı değil sınıf** yazar, **iş türüne göre** kurulur, **aşma yolunu** tanımlar.

DIKKAT

Bu tabloyu `copilot-instructions.md` 'ye koymayın. Maliyeti düşürmek için yazdığınız tablo her isteğe eklenerek maliyeti artırır. Wiki'ye koyun.

4.4 Gerçek işte önce/sonra ölçün

	ÖNCE	SONRA
Kullanılan model		
Bağlamdaki dosya sayısı		
Harcanan kredi		
Cevap kabul edilebilir mi?		

KALITE DÜŞTÜYSE NE YAPMALI

Sırayla deneyin: **1)** bağlamı aynı bırakın, yalnızca modeli yükseltin. **2)** model aynı kalsın, yalnızca bir dosya ekleyin.

Bu ikisini **ayrı ayrı** yapın. Aynı anda ikisini birden geri alırsanız hangisinin gerektiğini asla öğrenemezsiniz — ve gereksiz olanı sonsuza kadar ödersiniz.

4.5 Eylem planınızı yazın

	PLANINIZ
Yarın uygulayacağım kural	
Bu çeyrekte yapılacak değişiklik	
Hâlâ ihtiyacım olan veri	

Kendinize sorun

1. Yedi kaldıraçtan hangisi **kural olarak dayatılabilir**, hangisi ancak alışkanlıkla yerleşir?
2. Dayatılabilenler için mekanizma ne — politika mı, varsayılan ayar mı, eğitim mi?

EK

Referans kartı

Günün tamamı tek sayfada

DOSYALAR	
`copilot-instructions.md`	Her istekte · 500 token altı
`*.instructions.md`	`applyTo` eşleşince (IntelliJ'de yok)
`*.prompt.md`	`/komut` ile
`*.chatmode.md`	Mod seçilince (IntelliJ'de yok)

> **Kural:** her soruda gerekli değilse, her istekte yüklenen dosyaya girmez.

MCP	
Yapılandırma	`.vscode/mcp.json` veya IDE ayarları
Agent modu	Şart — Ask modunda araçlar çalışmaz
Desteklenen	Yalnızca tools
Tavan	İstek başına 128 araç

> **Kural:** yerel içeride demek değil. Kullanmadığınızı kapatın.

MALIYET	
Ölçülen	Girdi + çıktı + önbellek
Birim	1 AI kredisi = 1 sent
Dahil kredi	$19 Business / $39 Enterprise · havuzlanır
Ölçülmeyen	Kod tamamlama

> **Kural:** Billing → Usage'dan ölçün, asla tahmin etmeyin.

ÜÇ EŞIK	
Model seçimi	**25×**
Önbellek	**10×** ucuz (yazma maliyeti ayrı)
272Kuzun bağlam	**2×** pahalı

> **Kaldıraçlar çarpılıyor:** 25× ile 9× birlikte **224**× ediyor

EK

Yarın uygulayabileceğiniz üç şey

İzin gerektirmeyen üç değişiklik

1. Satır içi tamamlamanın halledebileceği işi sohbete taşımayı bırakın — izin gerekmiyor.
2. Yeni sohbeti hafif modelle açın; yetmezse yükseltin.
3. Konu değiştiğinde yeni sohbet açın.

NOT

Bu kitaptaki maliyet rakamları modellenmiş kullanım varsayımlarına dayanıyor ve büyüklük mertebesi göstermek içindir. Kendi gerçek rakamınız her zaman **GitHub → Settings → Billing → Usage** ekranındadır.

KÜNYE

Bu kitap hakkında

Kaynaklar, doğrulama ve notlarınız için yer

EĞITIM

Copilot, MCP ve Token Ekonomisi
Bir günlük atölye

KURUM

İstanbul Data Science Academy

EĞITMEN

Zekeriya Beşiroğlu
Zekeriya.besiroglu@istdsa.com

LAB MATERYALI

Katılımcı reposu
MCP sunucusu, maliyet hesaplayıcısı, 28 test

RAKAMLAR HAKKINDA

Bu kitaptaki maliyet senaryoları **modellenmiş kullanım varsayımlarına** dayanıyor; kanıt değil, büyüklük mertebesi göstergesi. Kendi gerçek rakamınız her zaman **GitHub → Settings → Billing → Usage** ekranındadır.

Model fiyatları değişiyor. Kendi kurumunuz için hesap yaparken güncel fiyat tablosunu GitHub Docs üzerinden doğrulayın.

NOTLARIM